

Teste de Software

Do Requisito ao Caso de Teste

Prof. Cleiton Tavares

Quanto Vale um Bug?



Therac-25 — 1985 a 1987



- Máquina de radioterapia computadorizada
 - Falhas de software que provocaram overdoses maciças de radiação em pacientes com câncer
- Quando a segurança nunca virou requisito testável
 - Uma condição de corrida no software permitia que edições rápidas do operador ignorassem interlocks de segurança que, em modelos anteriores, existiam fisicamente no hardware. O software nunca teve essa verificação testada de fato.

“Aqui o bug não custou dinheiro. Custou vidas.”



≥ 3 mortes

confirmadas, em 6 acidentes

Fonte: Leveson & Turner, “An Investigation of the Therac-25 Accidents”, IEEE Computer, 1993.

Ariane 5 — 4 de junho de 1996



- O foguete que explodiu por causa de código reaproveitado
 - Um módulo do Ariane 4 foi reaproveitado sem ser revalidado contra os requisitos de voo do Ariane 5. Um valor de velocidade horizontal (64 bits) foi convertido para um inteiro de 16 bits e estourou.

“O código funcionava perfeitamente — para os requisitos do foguete errado.”

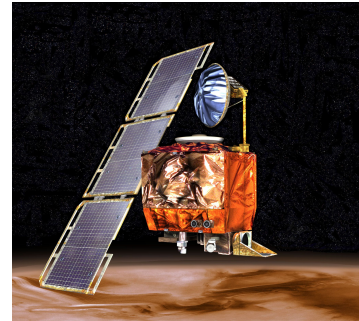


US\$ 370 mi

perdidos em 37 segundos

Fonte: Comissão de Inquérito Ariane 501 (ESA / Lions Report), 1996.

Mars Climate Orbiter — 23 de setembro de 1999



- Uma equipe usou libras-força. A outra, newtons.
 - A Lockheed Martin calculou o empuxo dos propulsores em libras-força; a JPL/NASA esperava newtons. A especificação de interface nunca foi validada por um teste de integração de ponta a ponta.

“Duas equipes, duas unidades, nenhum teste de integração pegou a diferença.”



US\$ 327,6 mi

de missão perdidos no espaço

Fonte: NASA Mars Climate Orbiter Mishap Investigation Board, 1999.

Knight Capital Group — 1º de agosto de 2012



- Empresa americana de serviços financeiros e formadora de mercado (market maker)
- Código de teste esquecido em produção
 - Uma rotina de teste obsoleta (“Power Peg”) ficou em um dos 8 servidores. Uma flag reaproveitada a reativou em produção, disparando ~4 milhões de ordens erradas. Nenhuma verificação cobriu a configuração completa do deploy.

“45 minutos para perder o equivalente a uma vida inteira de receita.”



US\$ 440–460 mi

Fonte: SEC (Securities and Exchange Commission), Comunicado 2013-222.

perdidos em 45 minutos

CrowdStrike — 19 de julho de 2024



- Isso não é história antiga
 - Uma empresa americana líder em tecnologia de segurança cibernética, conhecida por suas soluções nativas da nuvem
 - Uma atualização de “conteúdo” (não tratada com o mesmo rigor de teste do código) passou pelo validador da CrowdStrike sem ser barrada, causando leitura de memória fora dos limites em sistemas Windows no mundo todo.

“2024. Não é história do passado — é aviso do presente.”



US\$ 5,4 bi

e 8,5 milhões de PCs travados

Fonte: relatório de causa-raiz da CrowdStrike; estimativa de perdas Fortune 500 por Parametrix Insurance, 2024.

Nenhum deles foi “só um bug de código”

- Todos são lacunas entre o que deveria acontecer (o requisito) e o que foi de fato verificado (o teste).
- Em todos os casos, um cenário de teste bem pensado, e pensado cedo, poderia ter revelado o problema antes do lançamento.
- O custo de achar isso depois, em produção, no ar, em uso, é ordens de grandeza maior do que achar antes.

Custos com BUG

- US\$ 59,5 bilhões
 - por ano
 - é o que bugs de software custam à economia dos EUA

Mais de 1/3 desse custo seria evitável com infraestrutura de teste melhor, encontrando defeitos mais cedo no processo de desenvolvimento.

Fonte: NIST / RTI, “The Economic Impacts of Inadequate Infrastructure for Software Testing”, 2002.

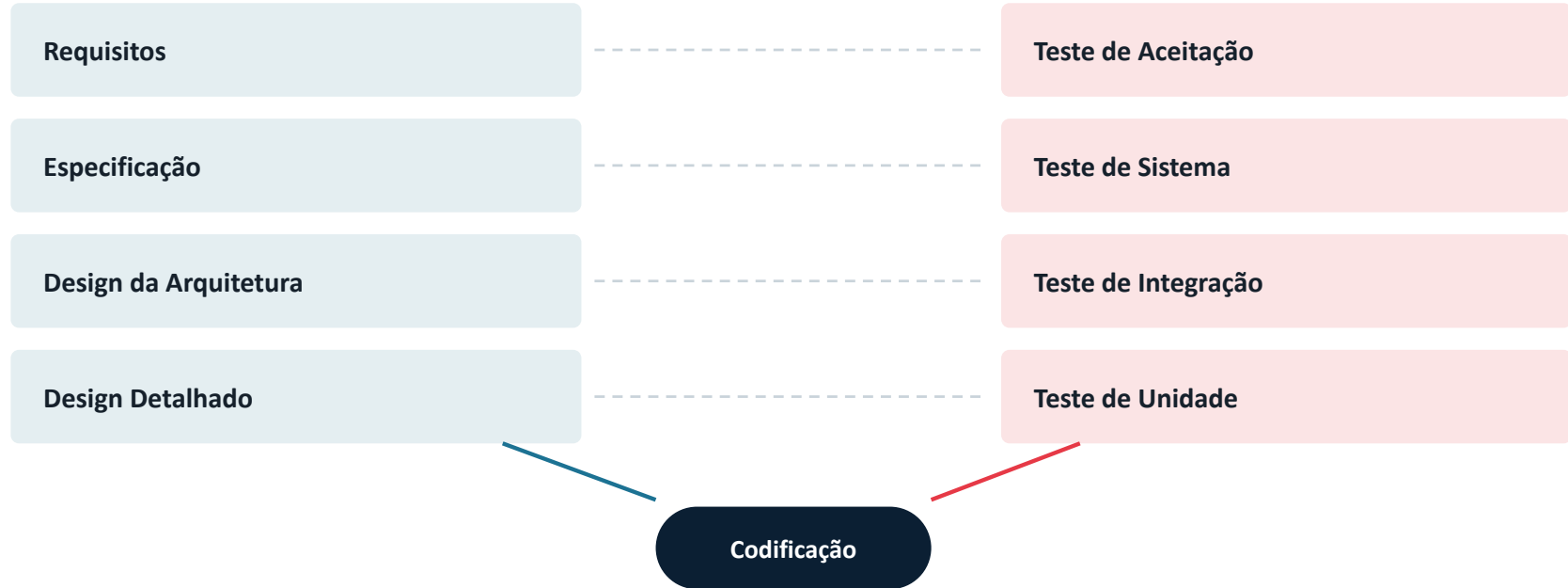
Testar Cedo

- “Testar cedo”, na prática, significa:
 - Pensar em cenários de teste desde o documento de requisitos
 - não esperar o código pronto para começar a pensar em teste.

O Modelo V

DEFINIÇÃO

VERIFICAÇÃO



Do requisito ao dado de teste



Requisito, cenário, caso e suíte não são sinônimos



Requisito

O comportamento esperado do sistema, descrito na especificação.



Cenário de teste

Uma situação de alto nível a ser verificada (ex.: “cupom expirado”).



Caso de teste

A concretização de um cenário: pré-condição, passos, dados e resultado esperado.



Suíte de teste

Um conjunto de casos de teste agrupados por objetivo, módulo ou execução.

O que torna um requisito testável

- **Específico**

- Descreve um comportamento concreto, não uma intenção vaga.

- **Verificável**

- É possível observar um resultado e dizer “passou” ou “falhou”.

- **Com critério de aceite**

- Deixa claro qual é o resultado esperado, inclusive nos limites.

- **Sem ambiguidade**

- Não permite duas leituras diferentes por duas pessoas diferentes.

“O sistema deve ser rápido.”

- Rápido para quem? Rápido em qual cenário? Rápido comparado a quê?
 - Dev A entende: “responde em até 2 segundos, no meu notebook.”
 - QA entende: “responde em até 2 segundos, com 1000 usuários simultâneos.”
 - Cliente entende: “mais rápido que o sistema antigo, em qualquer situação.”
- Versão testável
 - A página de resultados deve carregar em até 2 segundos, com até 1000 usuários simultâneos, em 95% das requisições.

Quatro perguntas para começar a pensar em teste



Caminho feliz

Qual é o uso normal, esperado, do requisito?



O que pode dar errado

Quais entradas inválidas ou falhas o sistema precisa tratar?



Limites e extremos

O que acontece exatamente na fronteira de uma regra?



O que NÃO deveria acontecer

Existe algo que o sistema precisa impedir explicitamente?

A matriz de rastreabilidade

- Uma ferramenta simples: cada requisito aponta para os casos de teste que o cobrem, e cada caso aponta de volta para o requisito.

	CT-01	CT-02	CT-03	CT-04	CT-05
RF-010 — Cupom de desconto	✓	✓	✓		✓
RF-011 — Cancelamento de pedido		✓		✓	
RF-012 — Login do cliente	✓		✓		

Carrinho de compras com cupom de desconto

- RF-010 — Aplicação de Cupom de Desconto
 - O sistema deve permitir que o cliente aplique um cupom de desconto ao carrinho de compras. O desconto só deve ser aplicado se o valor total do carrinho for igual ou superior ao valor mínimo definido pelo cupom. Cada cupom possui uma data de validade e um limite de uso por cliente.

Vamos raciocinar juntos



Caminho feliz

Carrinho acima do valor mínimo + cupom válido → desconto aplicado.



O que pode dar errado

Cupom expirado, ou já usado pelo cliente.



Limites e extremos

Carrinho com valor EXATAMENTE igual ao mínimo do cupom.



O que não deveria acontecer

Aplicar dois cupons na mesma compra — o requisito fala disso?

Seis cenários derivados do RF-010

01 Cupom válido, carrinho acima do mínimo

02 Carrinho abaixo do valor mínimo do cupom

03 Cupom expirado

04 Cupom já utilizado pelo cliente (limite atingido)

05 Carrinho com valor EXATAMENTE igual ao mínimo



06 Dois cupons aplicados na mesma compra



Todo caso de teste precisa responder 4 perguntas



Pré-condição

O que precisa ser verdade antes de começar?



Passos

O que o testador (ou script) faz, em ordem?



Dados de entrada

Quais valores concretos são usados?



Resultado esperado

O que define “passou” de forma inequívoca?

CT-005 — Valor do carrinho exatamente no mínimo

Pré-condição	Cliente logado; cupom “DESC10” válido; valor mínimo definido = R\$ 100,00
Passos	1) Adicionar produto de R\$ 100,00 ao carrinho. 2) Aplicar o cupom DESC10.
Dados de entrada	Cupom: DESC10 Valor do carrinho: R\$ 100,00 (exato)
Resultado esperado	? — o requisito diz “igual ou superior”, então deveria aplicar o desconto

Dicas

- Um bom caso de teste não só verifica o sistema
 - ele expõe requisitos malfeitos antes que virem código.
- Os cenários 05 (limite exato) e 06 (dois cupons) só existem porque perguntamos “e se...?” antes de escrever qualquer linha de código. Sem isso, essas decisões teriam sido tomadas, sem querer, por quem escreveu o código, no meio da implementação.

Agora em grupo

- Proponha cenários de teste para o próximo RF
- RF-011
 - O sistema deve permitir o cancelamento de um pedido pelo cliente em até 24 horas após a confirmação da compra, desde que o pedido ainda não tenha sido despachado para entrega.

Referências

- Comissão de Inquérito Ariane 501 (ESA / Lions Report), 1996.
- NASA Mars Climate Orbiter Mishap Investigation Board, Phase I Report, 1999.
- U.S. Securities and Exchange Commission, Release 2013-222 (Knight Capital), 2013.
- LEVESON, N.; TURNER, C. An Investigation of the Therac-25 Accidents. IEEE Computer, 1993.
- CrowdStrike, Root Cause Analysis (Channel File 291 Incident), 2024; Parametrix Insurance, 2024.
- NIST / RTI International. The Economic Impacts of Inadequate Infrastructure for Software Testing, 2002.
- BOSSAVIT, L. The Leprechauns of Software Engineering, 2015 (sobre a origem incerta da “curva de custo IBM”).
- BOEHM, B. Software Engineering Economics. Prentice Hall, 1981.



Obrigado